

وزارة التربية والتعليم العالي
مديرية التربية والتعليم: جنوب الخليل
المدرسة: دور الثانوية المهنية
التخصص: تطبيقات الهواتف الذكية

عنوان المشروع:

جذور فلسطين

اسم الطالب: منتصر عبد الكريم جبر

رقم الجلوس: 27057314

اسم المعلم المشرف: م. مؤيد حريبات

قدم هذا المشروع للحصول على شهادة الكفاءة المهنية

2026 / 2025

الفهرس

1 الفهرس

2 شكر وعرفان

3 ملخص المشروع

4 فكرة المشروع والمحتوى التعليمي

5 خطة المشروع

6 تنفيذ المشروع

7 واجهات التصميم

8 التوصيات

9 الصعوبات والتحديات

10 الخاتمة

11 المراجع والمصادر

12 الملاحق (الشفرات المصدريّة)

شكر وعرفان

الحمد لله رب العالمين، والصلاة والسلام على أشرف المرسلين سيدنا محمد وعلى آله وصحبه أجمعين
أتقدم بجزيل الشكر والامتنان إلى كل من ساعدني وعضدني في إنجاز هذا المشروع، وأخص بالشكر الد
الذي لم يدخر جهداً في تقديم النصائح والتوجيهات خلال فترة إعداد المشروع.
كما أتقدم بالشكر إلى إدارة المدرسة والمعلمين الكرام على جهودهم المبذولة في دعم الطالب وتشجيعه
الإبداع والتطوير.
ولا أنسى فضل الأسر الكريمة التي كانت الداعم الأول لي خلال فترة تنفيذ المشروع، فلهم دور كبير في
على الاستمرار والمثابرة على العمل الجاد.
وفي الختام، أسأل الله أن يجعل هذا العمل خالصاً لوجهه الكريم، وأن ينفع به الجميع.
والحمد لله رب العالمين.

ملخص المشروع

يُعد تطبيق "جذور فلسطين" منصة رقمية تفاعلية تهدف إلى توحيد وتوثيق المعلومات الخاصة بالمواقع ا في فلسطين، والتي كانت متفرقة سابقاً بين مصادر متعددة يصعب الوصول إليها، وفي فضاء رقمي شا

يعتمد التطبيق على معمارية برمجية حديثة (Clean Architecture) مع واجهة MVVM، ويركز على كفاءة المحلي وضمان استمرارية الخدمة حتى في المناطق ضعيفة الاتصال بنظام Offline-First. ويدعم التطبيق (عربية وإنجليزية) مع وضوح وسهولة للاستخدام، واكتمال الخرائط التفاعلية دون الحاجة لاتصال دائم بالإ

يجمع المشروع بين الجودة الهندسية والقيمة الثقافية، مقدماً تجربة استكشاف رقمية متكاملة للسائح و والمهتم، بالتراث الفلسطيني.

فكرة المشروع والمحتوى التعليمي

فكرة المشروع:

توفير قاعدة بيانات محلية شاملة ومحدثة لـ 30 موقع أثري موزعة على 10 محافظات فلسطينية، مع واجهة للمستخدم البحث والتصفية، التنقل الخرائطي، حفظ المواقع المفضلة، كل ذلك دون الحاجة لاتصال دائم

المحتوى التعليمي والمهارات المكتسبة:

- فصل طبقات البيانات والمنطق وواجهة MVVM ونمط تطبيق Clean Architecture.
- استخدام Room Database للتخزين الهيكلي وإدارة العمليات غير المتزامنة.
- ضمان الأمان والاستقرار عبر ViewBinding لربط الواجهات بـ XML.
- إدارة حالة التطبيق وتدفق البيانات باستخدام Coroutines و Flow.
- دمج Google Maps SDK لعرض الإحداثيات الجغرافية بدقة.
- دعم الوضع الليلي/النهاري عبر تطبيق Material Components.
- الحقن اليدوي للتبعيات (Manual DI) باستخدام ViewModelProvider.Factory لضمان قابلية الاختبار.
- دعم تعدد اللغات (عربي/إنجليزي) باستخدام LocaleHelper مع حفظ التفضيل عبر SharedPreferences.
- استخدام SharedPreferences لإدارة إعدادات المستخدم (اللغة، الوضع الليلي، حالة التعريف) مع التغيرات عبر Flow.
- استخدام Gson لتحليل بيانات JSON وتحويلها إلى كائنات Kotlin.

خطة المشروع

مراحل تنفيذ المشروع:

المرحلة	الوصف
الأولى	تحليل المتطلبات: جمع بيانات المواقع الأثرية، وتصميم هيكل قاعدة البيانات
الثانية	تصميم واجهات المستخدم باستخدام XML و Material Components
الثالثة	إعداد بيئة التطوير وربط المكتبات (Room, Coroutines, Gson, ViewBinding)
الرابعة	برمجة طبقة البيانات وتنفيذ نمط MVVM، وربط الواجهات بالمنطق البرمجي
الخامسة	دمج الميزات المتقدمة: البحث والتصفية، الوضع الليلي، نظام المفضلة، خرائط Google، وتعدد اللغات
السادسة	الاختبار الشامل: على أجهزة حقيقية، إصلاح الأخطاء، تحسين الأداء
السابعة	التوثيق الفني: إعداد التقرير النهائي، والتحضير للعرض أمام اللجنة

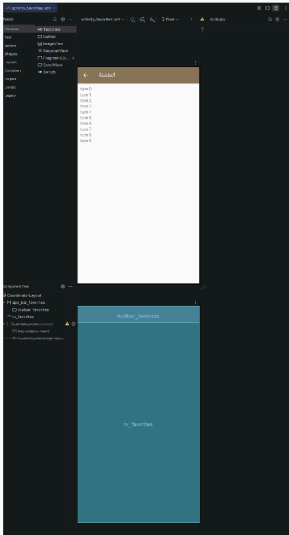
تنفيذ المشروع

تم تنفيذ المشروع بخطوات منهجية وفق دورة حياة تطوير البرمجيات (SDLC):

1. تحليل البيانات: جمع المعلومات التاريخية والجغرافية للمواقع، وهيكلية ملف JSON بتسمية موحدة.
2. إعداد بيئة المشروع: تهيئة Android Studio، وربط Gradle مع إصدارات مستقرة (JDK 11، 7.4.2، compileSdk 33).
3. إنشاء كيان البيانات: بناء طبقة البيانات Room مع واجهة DAO، وتفعيل Gson لقراءة البيانات الأولية مع استخدام @SerializedName لربط أسماء حقول JSON بأسماء أعمدة قاعدة البيانات بدقة.
4. تطبيق MVVM: إنشاء ViewModel مع عزل المنطق عن العرض، وربط البيانات باستخدام LiveData.
5. تصميم شاشات التطبيق: تطوير الواجهات عبر XML، وتفعيل ViewBinding لربط العناصر بأمان.
6. إضافة الميزات المتقدمة: دمج GoogleMap للعرض الجغرافي، وتفعيل البحث والتصفية، ونظام الـ SharedPreferences، ودعم تعدد اللغات (عربي/إنجليزي) باستخدام LocaleHelper.
7. الاختبار والتحسين: تشغيل على أجهزة حقيقية، معالجة الأخطاء، وتحسين الأداء.

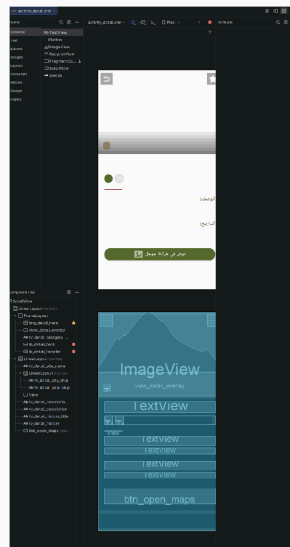
ملاحظة: يتم استخدام الحقن اليدوي للتبعيات (Manual Dependency Injection) عبر ViewModelProvider.Factory لإنشاء ViewModel مع تمرير التبعيات (Repository و DataManager) بدلاً من استخدام إطار عمل Hilt، مما يضمن بساطة الكود وسهولة الفهم مع الحفاظ على قابلية الاختبار.

واجهات التصميم



واجهة المفضلة (activity_favorites.xml)

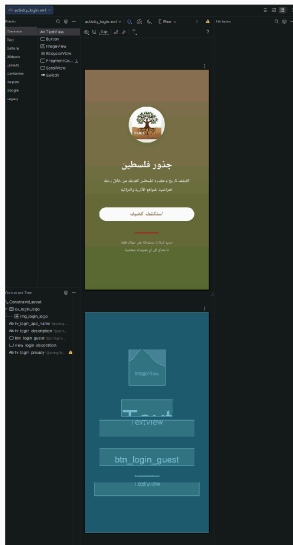
واجهات التصميم



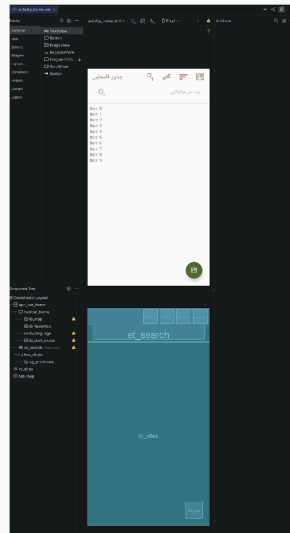
واجهة تفاصيل الموقع (activity_detail.xml)

واجهة المفضلة (activity_favorites.xml)

واجهة تفاصيل الموقع (activity_detail.xml)



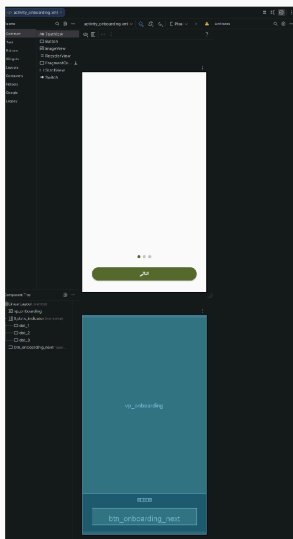
واجهة تسجيل الدخول (activity_login.xml)



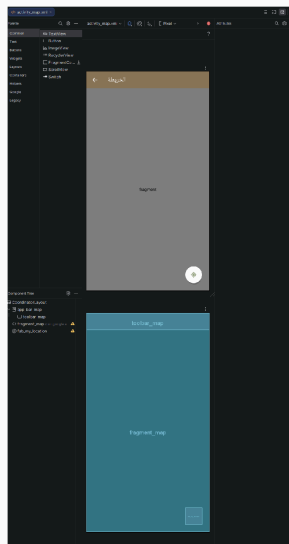
واجهة الصفحة الرئيسية (activity_home.xml)

واجهة تسجيل الدخول (activity_login.xml)

واجهة الصفحة الرئيسية (activity_home.xml)



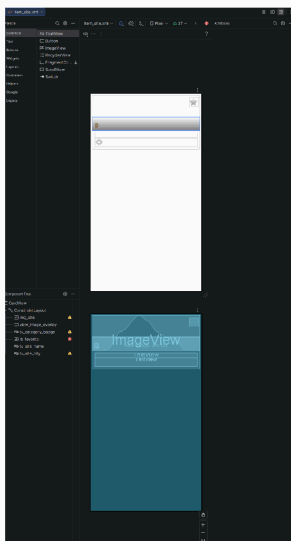
واجهة التعريف بالتطبيق (activity_onboarding.xml)



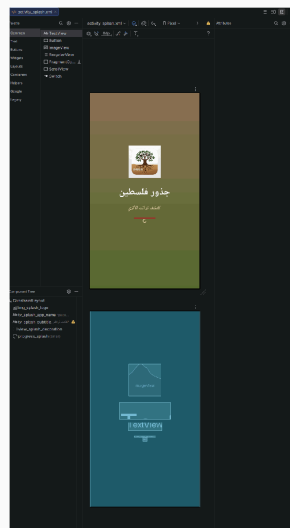
واجهة الخريطة التفاعلية (activity_map.xml)

واجهة تعريف بالتطبيق (activity_onboarding.xml)

واجهة الخريطة التفاعلية (activity_map.xml)



عنصر الموقع (item_site.xml)



واجهة شاشة البداية (activity_splash.xml)

عنصر الموقع (item_site.xml)

واجهة شاشة بداية (activity_splash.xml)

التوصيات

- ربط التطبيق بالسحابة: إضافة Firebase لتحديث البيانات تلقائياً عند توفر الإنترنت، مما يتيح إضافة دون تحديث التطبيق.
- دعم الواقع المعزز (AR): عرض نماذج ثلاثية الأبعاد للمواقع الأثرية باستخدام ARCore لتجربة غامرة.
- تمكين المستخدمين من مشاركة تجاربهم: أنظمة التقييم والتعليقات للمواقع، مما يشري المحتوى بتج الحقيقين.
- إضافة دعم للغات إضافية: توسيع نطاق اللغات ليشمل (الفرنسية، الإسبانية، العبرية) لتقديم التطبيق أوسع من المستخدمين حول العالم.
- نشر المشروع كمصدر مفتوح: رفع الكود على GitHub لتشجيع المطورين على المساهمة في تطوير وتحسينه.
- تحسين نظام اللغات: إضافة المزيد من النصوص المترجمة وضمان توافق جميع الواجهات مع اتجاه (RTL/LTR) عند التبديل بين العربية والإنجليزية.

الصعوبات والتحديات

1. تعارض تسمية حقول JSON مع كود Kotlin

الأثر: اختفاء بيانات المواقع عند التحليل

الحل: إنشاء ربط دقيق بين التسميتين باستخدام SerializedName@ (مثل foundationYear, image_url ↔ foundation_year ↔).

2. أعطال مفاجئة أثناء التنقل بين الشاشات

الأثر: تعطل التطبيق (Crash)

الحل: تفعيل ViewBinding بشكل كامل مع إدارة دورة حياة Lifecycle، والتأكد من استيراد Context و Intent الأنشطة.

3. تشتت إعدادات المستخدم

الأثر: فقدان التفضيلات عند إعادة التشغيل (اللغة، الوضع الليلي)

الحل: استخدام SharedPreferences عبر PreferencesManager مع مراقبة المتغيرات عبر (callbackFlow لضمان استجابة فورية لتغيرات الإعدادات).

4. توافق بيئة التطوير (JDK, Gradle, AGP)

الأثر: فشل عملية البناء

الحل: توحيد الإصدارات عبر compileOptions و (JDK 11, AGP 7.4.2, compileSdk 33) kotlinOptions، وإضافة android.overridePathCheck=true لمعالجة مشكلة المسار العربي.

5. تحميل الصور من المسار المحلي

الأثر: بطء التحميل وارتفاع استهلاك RAM

الحل: استخدام Glide مع Disk/Memory Cache لتحسين أداء تحميل الصور وإدارتها بكفاءة.

6. دعم تعدد اللغات في وقت التشغيل

الأثر: عدم استجابة التطبيق لتغيير اللغة بدون إعادة تشغيل

الحل: إنشاء LocaleHelper لتطبيق اللغة عبر attachBaseContext في كل نشاط، مع حفظ التفضيل في SharedPreferences وإعادة تحميل النشاط (recreate) عند التبديل.

الخاتمة

نجد مشروع "جذور فلسطين" في تحقيق هدفه الأساسي المتمثل في توحيد وعرض التراث الأثري الفلد منصة رقمية واحدة، مستقلة وسهلة الاستخدام.

تم ضمان كود نظيف، قابل للصيانة، وقابل للتوسع، وذلك من خلال اعتماد معمارية Clean Architecture وأثبت التطبيق قدرته على العمل بكفاءة دون اتصال بالإنترنت، مع دعم تعدد اللغات (عربي/إنجليزي) وال

يمثل هذا المشروع خطوة عملية نحو رقمنة التراث الفلسطيني، ويعكس القدرة التقنية على دمج أدوات الحديثة مع الهوية الثقافية المحلية، مما يتيح للجيل القادم فرصة التعرف على تاريخه وثقافته بأسلوب ت

المراجع والمصادر

1. Google Developers. (2024). Android Developers Documentation: Kotlin, Room, ViewBinding, Coroutines. developer.android.com
2. JetBrains. (2024). Kotlin Programming Language Reference. kotlinlang.org
3. Google Maps Platform. (2024). Maps SDK for Android Documentation. developers.google.com/maps
4. Material Design. (2024). Material Components for Android Guidelines. m3.material.io
5. وزارة السياحة والآثار الفلسطينية. قاعدة بيانات المواقع الأثرية في الضفة الغربية.
6. المراجع التاريخية المحلية ومواقع التراث العالمي (UNESCO).
7. مجتمعات المطورين: Stack Overflow, GitHub, Android Developers Blog.

الملاحق

الشفرات المصدريّة

كود LocaleHelper.kt - مساعد اللغة

```
m.palestine.roots.util

roid.content.Context
roid.content.res.Configuration
a.util.Locale

aleHelper {

e const val PREFS_NAME = "settings"
e const val KEY_LANGUAGE = "language"

tSavedLanguage(context: Context): String {
l prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
turn prefs.getString(KEY_LANGUAGE, "ar") ?: "ar"

polyLocale(context: Context): Context {
turn setLocale(context, getSavedLanguage(context))

tLocale(context: Context, language: String): Context {
l locale = Locale(language)
cale.setDefault(locale)
l config = Configuration(context.resources.configuration)
nfig.setLocale(locale)
nfig.setLayoutDirection(locale)
turn context.createConfigurationContext(config)
```


كود PreferencesManager.kt - إدارة التفضيلات

```

m.palestine.roots.data.local

android.content.Context
android.content.SharedPreferences
kotlinx.coroutines.flow.Flow
kotlinx.coroutines.flow.callbackFlow
kotlinx.coroutines.channels.awaitClose

class PreferencesManager(private val context: Context) {

    private val prefs: SharedPreferences =
        context.getSharedPreferences("settings", Context.MODE_PRIVATE)

    // Configuration object {
    private const val KEY_DARK_MODE = "dark_mode"
    private const val KEY_LANGUAGE = "language"
    private const val KEY_ONBOARDING_COMPLETED = "onboarding_completed"

    // Dark Mode
    val DarkMode: Flow<Boolean> = callbackFlow {
        val listener = SharedPreferences.OnSharedPreferenceChangeListener { _, key ->
            if (key == KEY_DARK_MODE) trySend(prefs.getBoolean(KEY_DARK_MODE, false))
        }

        prefs.registerOnSharedPreferenceChangeListener(listener)
        trySend(prefs.getBoolean(KEY_DARK_MODE, false))
        awaitClose { prefs.unregisterOnSharedPreferenceChangeListener(listener) }
    }

    // Language
    val Language: Flow<String> = callbackFlow {
        val listener = SharedPreferences.OnSharedPreferenceChangeListener { _, key ->
            if (key == KEY_LANGUAGE) trySend(prefs.getString(KEY_LANGUAGE, "ar") ?: "ar")
        }

        prefs.registerOnSharedPreferenceChangeListener(listener)
        trySend(prefs.getString(KEY_LANGUAGE, "ar") ?: "ar")
        awaitClose { prefs.unregisterOnSharedPreferenceChangeListener(listener) }
    }

    // Set Language
    fun setLanguage(lang: String) {
        prefs.edit().putString(KEY_LANGUAGE, lang).apply()
    }

    // Toggle Dark Mode
    fun toggleDarkMode(enabled: Boolean) {
        prefs.edit().putBoolean(KEY_DARK_MODE, enabled).apply()
    }

    // Set Onboarding Completed
    fun setOnboardingCompleted() {
        prefs.edit().putBoolean(KEY_ONBOARDING_COMPLETED, true).apply()
    }
}

```

كود SiteEntity.kt - كيان قاعدة البيانات

```

m.palestine.roots.data.local.entity

roidx.room.ColumnInfo
roidx.room.Entity
roidx.room.PrimaryKey
.google.gson.annotations.SerializedName

pleName = "sites")
    SiteEntity(
ryKey
lizedName("id")
: String,

lizedName("name")
me: String,

nInfo(name = "name_en")
lizedName("name_en")
meEn: String,

lizedName("city")
ty: String,

nInfo(name = "city_en")
lizedName("city_en")
tyEn: String,

lizedName("description")
scription: String,

nInfo(name = "description_en")
lizedName("description_en")
scriptionEn: String,

lizedName("history")
story: String,

nInfo(name = "history_en")
lizedName("history_en")
storyEn: String,

nInfo(name = "image_url")
lizedName("imageUrl")
ageUrl: String,

lizedName("latitude")
titude: Double,

lizedName("longitude")
ngitude: Double,

lizedName("category")
tegrity: String,

nInfo(name = "category_en")

```

```
lizedName("category_en")
categoryEn: String,

nInfo(name = "foundation_year")
lizedName("foundationYear")
oundationYear: String?,

nInfo(name = "is_favorite", defaultValue = "0")
lizedName("is_favorite")
Favorite: Boolean = false
```


كود HomeViewModel.kt - نموذج العرض الرئيسي

```

m.palestine.roots.viewmodel

androidx.lifecycle.ViewModel
androidx.lifecycle.ViewModelProvider
androidx.lifecycle.viewModelScope
m.palestine.roots.data.local.PreferencesManager
m.palestine.roots.domain.model.Site
m.palestine.roots.domain.repository.SiteRepository
kotlinx.coroutines.flow.*
kotlinx.coroutines.launch

class ViewModel(
    private val repository: SiteRepository,
    private val preferencesManager: PreferencesManager
) {

    private val _uiState = MutableStateFlow<UiState>(UiState.Loading)
    val uiState: StateFlow<UiState> = _uiState.asStateFlow()

    private val _searchQuery = MutableStateFlow("")
    val searchQuery: StateFlow<String> = _searchQuery.asStateFlow()

    val darkMode = preferencesManager.isDarkMode
    viewModelScope.launch(SharingStarted.WhileSubscribed(5000), false) {
        darkMode = preferencesManager.isDarkMode
    }

    val language = preferencesManager.language
    viewModelScope.launch(SharingStarted.WhileSubscribed(5000), "ar") {
        language = preferencesManager.language
    }

    data class Province(val ar: String, val en: String)

    val provinces = listOf(
        Province("القدس", "Jerusalem"),
        Province("رام الله والبيرة", "Ramallah & Al-Bireh"),
        Province("نابلس", "Nablus"),
        Province("جنين", "Jenin"),
        Province("طولكرم", "Tulkarem"),
        Province("قلقيلية", "Qalqilya"),
        Province("سلفيت", "Salfit"),
        Province("أريحا والأغوار", "Jericho & Jordan Valley"),
        Province("الخليل", "Hebron"),
        Province("بيت لحم", "Bethlehem")
    )

    fun getProvinceNames(lang: String): List<String> =
        provinces.map { if (lang == "en") it.en else it.ar }

    fun solveArabicCityName(displayName: String): String =
        provinces.find { it.ar == displayName || it.en == displayName }?.ar ?: displayName

    fun (loadAllSites, onSearchQueryChanged, filterByCity, toggleFavorite, toggleDarkMode,
        toggleLanguage)() {

        class UiState {
            object Loading : UiState()
            data class Success(val sites: List<Site>) : UiState()
        }
    }
}

```

```
data class Error(val message: String) : UiState()

class HomeViewModelFactory(
    private val repository: SiteRepository,
    private val preferencesManager: PreferencesManager
) : ViewModelProvider.Factory {
    @Suppress("UNCHECKED_CAST")
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        return HomeViewModel(repository, preferencesManager) as T
    }
}
```

كود HomeActivity.kt - النشاط الرئيسي

```

m.palestine.roots.ui

roid.content.Context
roid.content.Intent
roid.os.Bundle
roid.view.View
roidx.appcompat.app.AppCompatActivity
roidx.appcompat.app.AppCompatActivityDelegate
roidx.core.widget.addTextChangedListener
roidx.activity.viewModels
roidx.lifecycle.lifecycleScope
roidx.recyclerview.widget.GridLayoutManager
.google.android.material.chip.Chip
.palestine.roots.R
.palestine.roots.data.local.PreferencesManager
.palestine.roots.data.local.db.PalestineDatabase
.palestine.roots.data.repository.SiteRepositoryImpl
.palestine.roots.databinding.ActivityHomeBinding
.palestine.roots.util.LocaleHelper
.palestine.roots.viewmodel.HomeViewModel
linx.coroutines.flow.collectLatest
linx.coroutines.flow.first
linx.coroutines.launch

Activity : AppCompatActivity() {

    val binding by lazy { ActivityHomeBinding.inflate(layoutInflater) }
    val viewModel: HomeViewModel by viewModels {
        val dao = PalestineDatabase.getInstance(this).siteDao()
        val repo = SiteRepositoryImpl(dao)
        val prefs = PreferencesManager(this)
        HomeViewModel.Factory(repo, prefs) // Manual DI
    }

    de fun attachBaseContext(newBase: Context) {
        super.attachBaseContext(LocaleHelper.applyLocale(newBase))

        (setupRecyclerView, setupProvinceChips, setupSearch, setupToolbarActions,
        viewModel)
    }
}

```


كود DetailActivity.kt - نشاط تفاصيل الموقع

```
m.palestine.roots.ui

import android.content.Context
import android.content.Intent
import android.os.Bundle
import m.palestine.roots.util.LocaleHelper

class DetailActivity : AppCompatActivity() {
    private val viewModel: HomeViewModel by viewModels {
        HomeViewModel.Factory(
            dao = PalestineDatabase.getInstance(this).siteDao(),
            repo = SiteRepositoryImpl(dao),
            prefs = PreferencesManager(this)
        ) // Manual DI
    }

    override fun attachBaseContext(newBase: Context) {
        super.attachBaseContext(LocaleHelper.applyLocale(newBase))
    }

    override fun displaySite(site: Site) {
        val lang = currentLang
        binding.tvDetailSiteName.text = if (lang == "en") site.nameEn else site.name
        binding.tvDetailCityChip.text = if (lang == "en") site.cityEn else site.city
        binding.tvDetailDescription.text = if (lang == "en") site.descriptionEn else site.description
        binding.tvDetailHistory.text = if (lang == "en") site.historyEn else site.history
        binding.tvDetailCategoryBadge.text = if (lang == "en") site.categoryEn else site.category
    }
}
```

كود MapActivity.kt - نشاط الخريطة (مقتطف)

```
m.palestine.roots.ui

.palestine.roots.util.LocaleHelper

ctivity : AppCompatActivity(), OnMapReadyCallback {
de fun attachBaseContext(newBase: Context) {
per.attachBaseContext(LocaleHelper.applyLocale(newBase))

e fun addMarkers() {
l map = googleMap ?: return
p.clear()
tes.forEach { site ->
    val position = LatLng(site.latitude, site.longitude)
    val title = if (currentLang == "en") site.nameEn else site.name
    val snippet = if (currentLang == "en") site.cityEn else site.city
    val marker = map.addMarker(
        MarkerOptions().position(position).title(title).snippet(snippet)
    )
    marker?.tag = site.id
```

كود PalestineDatabase.kt - قاعدة البيانات


```

m.palestine.roots.data.local.db

android.content.Context
androidx.room.Database
androidx.room.Room
androidx.room.RoomDatabase
androidx.sqlite.db.SupportSQLiteDatabase
m.palestine.roots.data.local.dao.SiteDao
m.palestine.roots.data.local.entity.SiteEntity
com.google.gson.Gson
com.google.gson.reflect.TypeToken
kotlinx.coroutines.CoroutineScope
kotlinx.coroutines.Dispatchers
kotlinx.coroutines.launch
kotlinx.coroutines.withContext

entities = [SiteEntity::class], version = 4, exportSchema = false)
class PalestineDatabase : RoomDatabase() {

    object siteDao(): SiteDao

    companion object {
        @Volatile
        private var INSTANCE: PalestineDatabase? = null

        fun getInstance(context: Context): PalestineDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    PalestineDatabase::class.java,
                    "palestine_database"
                )
                    .fallbackToDestructiveMigration()
                    .addCallback(PopulateCallback(context.applicationContext))
                    .build()
                INSTANCE = instance
                instance
            }
        }

        fun PopulateCallback(private val context: Context) : RoomDatabase.Callback() {
            override fun onCreate(db: SupportSQLiteDatabase) {
                super.onCreate(db)
                CoroutineScope(Dispatchers.IO).launch { populateDatabase() }
            }

            override fun onOpen(db: SupportSQLiteDatabase) {
                super.onOpen(db)
                CoroutineScope(Dispatchers.IO).launch { ensureDataPopulated() }
            }

            ... (populateDatabase, ensureDataPopulated using Gson + assets)
        }
    }
}

```

